**A Project Report on**

# Automated Face Recognition Attendance System

Submitted in partial fulfillment for award of

**Bachelor of Technology**

Degree

in

## Computer Science and Engineering

By


**D.Nethra Sree(Y22ACS442)**

**A. Akhil (Y22ACS402)**

**K.Venkata Siva Naga Sai(Y22ACS472)**

**D.Rama Lakshman(Y22ACS447)**


Under the guidance of

**Dr. C. Prakasa Rao, M.Tech, Ph.D**

Professor


Department of Computer Science and Engineering

**Bapatla Engineering College**

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

**BAPATLA – 522102, Andhra Pradesh, INDIA**


**2025-2026**

**Department of**

**Computer Science and Engineering**

# CERTIFICATE

This is to certify that the project report entitled **Automated Face Recognition Attendance System** that is being submitted by

D. Nethra Sree (Y22ACS442), A. Akhil (Y22ACS402), K. Venkata Siva Naga Sai (Y22ACS472), D. Rama Lakshman (Y22ACS447) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

| | |
|---|---|
| **Signature of the Guide** | **Signature of the HOD** |
| **Dr. C. Prakasa Rao** | **Dr. M. Rajesh Babu** |
| **Professor** | **Assoc. Prof. & Head** |

# DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

D. Nethra Sree(Y22ACS442)

A. Akhil(Y22ACS402)

K. Venkata Siva Naga Sai(Y22ACS472)

D. Rama Lakshman(Y22ACS447)

# Acknowledgement

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide and, **Dr. C. Prakasa Rao, Professor**, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to **Project Co-coordinator Dr. P. Venkata Lakshmi, Associate Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

D. Nethra Sree(Y22ACS442)
A. Akhil(Y22ACS402)
K. Venkata Siva Naga Sai (Y22ACS472)
D. Rama Lakshman(Y22ACS447)

# ABSTRACT

The conventional pen-and-paper roll-call method of attendance recording is time-consuming, prone to human error, and easily manipulated through proxy attendance. This project presents the Automated Face Recognition Attendance System (AFRAS), an AI-powered web application that replaces manual attendance with a fully passive, camera-based identification pipeline. The system was declared with the face recognition Python library combined with OpenCV to capture live CCTV or webcam frames, compute 128-dimensional facial encoding vectors for each detected face, and identify students by Euclidean distance comparison against pre-stored encodings. Attendance is recorded period-wise into a SQLite relational database when the identified face is detected within a valid academic period window and on a confirmed working day. Public holidays are automatically synchronized using the Google Calendar iCal API. A Flask-based web portal with role-based access serves two portals. The HOD/Faculty portal for attendance monitoring, manual correction, departmental filtering, and Excel export. The student portal for viewing personal cumulative attendance percentage and period-wise history. The system supports dynamic student self-registration with live webcam image capture and encodes new students immediately without restarting the recognition engine. The proposed system achieves over 95% face recognition accuracy, processes 5–10 simultaneous faces per frame, and eliminates proxy attendance entirely.

**Keywords:** Face Recognition, HOG, OpenCV, Attendance System, Flask, SQLite, CCTV, Period-wise Attendance, Role-based Access, Google Calendar API, Python.

# Table of Contents

Abstract……...……...…….v

List of Tables……...viii

List of Figures……...ix

List of Symbols, Abbreviations……..x

1 INTRODUCTION…….…….…..…1

1.1 Motivation…….....……..……1

1.2 Problem Statement..….……..…….3

1.3 Objectives of the Project ……5

2 LITERATURE SURVEY…….……..…….....8

2.1 Existing Systems…….…….……...9

2.2 Disadvantages of Existing System……..……..10

2.3 Proposed System……....……..……12

2.4 Advantages of Proposed System...…….……...13

3 ANALYSIS…....…….……....…..…...…...…..……...16

3.1 Software Requirement Specification…….……16

3.1.1 User Requirements……..……16

3.1.2 Software Requirements…….……..17

3.1.3 Hardware Requirements……..……...18

3.2 Content Diagram of Project..…….……...19

4 DESIGN…….…… 21

4.1 Introduction……...21

4.2 Data Flow Diagram…….……..23

4.2.1 DFD Level 0…….23

4.2.2 DFD Level 1……..……24

4.2.3 ER Diagram……..…..……27

4.2.4 UML Use Case Diagram……..……..28

4.3 Data Dictionary…….30

5 IMPLEMENTATION & RESULTS……..…….……...34

5.1 Method of Implementation…….……..34

5.1.1 Implementation of Algorithms….…….34

5.1.2 Output Screens…….….……...36

5.1.3 Result Analysis…….……...…….40

6 TESTING & VALIDATION…….…….……...42

6.1 Features to be Tested…….…….42

6.2 Features Not to be Tested….…….……..……...43

6.3 Testing Tools and Environment…….……..…….……..43

6.4 Design of Test Cases and Scenarios…….……..44

7 CONCLUSION & FUTURE ENHANCEMENT….…….46

8 REFERENCES……...48

# List of Tables

Table 2.1 Comparison of Attendance Systems……...……11

Table 4.1 Data Dictionary — users table……...30

Table 4.2 Data Dictionary — period_attendance table……..31

Table 4.3 Data Dictionary — attendance table……..32

Table 4.4 Data Dictionary — holidays table……..32

Table 4.5 Period Configuration…….……33

Table.6.1 Development and Testing Tools Used……44

# List of Figures

Figure 1.1 Block Diagram of the Proposed System……..…….7

Figure 3.1 Content Diagram……..…….19

Figure 4.1 System Architecture of Attendance System…….…….……....22

Figure 4.2 DFD Level 0 — Context Diagram ……..……...…….23

Figure 4.3 DFD Level 1— Process Breakdown ……...……26

Figure 4.4 ER Diagram of the Attendance Database…….28

Figure 4.5 UML Use Case Diagram….…….29

Figure 5.1 Login Page of the Web Portal……..…….36

Figure 5.2 Student Signup page……..……...……37

Figure 5.3 Student Attendance Dashboard……37

Figure 5.4 HOD Dashboard…….…….38

Figure 5.5 Manual Attendance Page by HOD…….…….39

Figure 5.6 HOD Signup Page…….……..39

Figure 5.7 CCTV/Webcam Live Recognition Window……...…….40

# List of Symbols, Abbreviations

| Abbreviation / Symbol | Full Form |
|---|---|
| 1. CCTV | Closed-Circuit Television |
| 2. RTSP | Real-Time Streaming Protocol |
| 3. API | Application Programming Interface |
| 4. HOD | Head of Department |
| 5. IST | Indian Standard Time |
| 6. DB | Database |
| 7. SQLite | Serverless, file-based SQL Database |
| 8. LAN | Local Area Network |
| 9. DOB | Date of birth |
| 10. HOG | Histogram of Oriented Gradients |
| 11. ical | Internet Calendar format(.ics) |
| 12. PK / FK | Primary Key, Foreign Key |
| 13. ✓ / ✗ | Symbols for “yes” or “no” in comparison tables. |
| 14. P1 – P6 | Period 1 to period 6 |
| 15. % | Attendance percentage |
| 16. OD | On Duty |
| 17. RFID | Radio Frequency Identification |
| 18. xlsx | Microsoft Excel Open XML Format |
| 19. 0.0 | Attendance value for Absent period |
| 20. 1.0 | Attendance value for Present period |
| 21. 0.45 | Face recognition distance threshold |
| 22. UML | Unified Modeling Language |
| 23. AFRAS | Automated Face Recognition Attendance System |

# 1 INTRODUCTION

Maintaining accurate attendance records is critical in educational institutions for enforcing discipline and determining exam eligibility. In many colleges, attendance is taken manually by calling names or having students sign sheets, which consumes **5–10 minutes per class** and disrupts learning. Traditional methods also allow proxy attendance where students signing in for absent peers, and the resulting paper records are cumbersome to compile. Face recognition technology has advanced significantly with deep learning. Modern libraries (e.g. *face_recognition* with dlib) can perform highly accurate face identification in real time on standard hardware. By integrating such algorithms with existing CCTV cameras in classrooms, it becomes feasible to fully automate attendance without additional hardware.

The key problems with manual attendance are well-documented: it is time-consuming (wasting class time for roll-calls), error-prone, and vulnerable to fraud (proxy attendance). Moreover, manual methods offer no real-time visibility of attendance data for students or administrators. There is also typically no automatic handling of non-working days (weekends or public holidays), which can skew attendance calculations. In summary, the lack of automation leads to delays, inaccuracies, and administrative overhead. This motivates the development of a computer vision–based system that automatically recognizes student faces from live video and marks attendance instantly, thereby resolving all the aforementioned issues.

## 1.1 MOTIVATION

Attendance monitoring is a fundamental and legally mandated academic requirement in every engineering institution affiliated with universities across India. Under the University Grants Commission (UGC) guidelines and most university regulations, a student must maintain a minimum attendance percentage — typically 75% of the total number of periods conducted — to be eligible to appear for semester-end examinations. Failure to meet this minimum threshold can result in detention, preventing a student from appearing in examinations regardless of their academic performance on internal assessments. It is therefore not merely a bureaucratic formality but a critical academic performance indicator that directly determines a student's progression through their degree program.

Despite this importance, the prevailing method of attendance recording in the vast majority of engineering colleges remains archaic and manual. Faculty members physically call out each student's name from a printed register at the beginning of every class, or pass around a paper sheet for students to sign. In a typical engineering classroom with 60 to 80 students enrolled, this manual roll-call process consumes anywhere between 10 and 15 minutes per period. With six periods per working day and approximately 200 working days in an academic year, this translates to a staggering loss of instructional time — time that could otherwise be devoted to teaching and learning.

Beyond the time inefficiency, the manual system suffers from a well-known and widely exploited vulnerability: proxy attendance. A student who is physically absent from class can have a friend or classmate mark attendance on their behalf, either by calling out "present" during verbal roll call or by signing the attendance sheet for the absent student. This practice undermines the very purpose of attendance monitoring, renders the recorded data unreliable, and can have serious academic and disciplinary consequences if left unchecked. Studies in Indian engineering institutions have revealed that proxy attendance rates in large classrooms can be as high as 10–15% of the daily attendance records, making cumulative attendance data significantly inaccurate.

Furthermore, the administrative burden placed on faculty extends well beyond the classroom. After manually recording roll calls on paper, faculty are required to periodically transfer attendance data into digital spreadsheets or institutional ERP portals. At the end of each month or before examination cycles, attendance percentages must be computed for each student across multiple subjects — a labour-intensive and repetitive process involving considerable manual arithmetic and data entry, all of which is susceptible to calculation errors, transcription mistakes, and data loss due to misplaced or damaged paper registers.

The emergence of affordable high-resolution CCTV cameras, powerful open-source computer vision libraries (such as OpenCV), and accessible machine learning frameworks (such as the face recognition4 library and dlib) has created a compelling technological opportunity to address all of these challenges simultaneously. Facial recognition technology, once limited to government and defense applications

requiring expensive specialized hardware, is now mature and accurate enough to run in real time on a standard desktop computer using nothing more than a USB webcam or a network-connected IP camera.

The primary motivation for this project is therefore to harness these modern technological capabilities to develop an intelligent, automated attendance system that can silently and passively identify students from live classroom video feeds, record period-wise presence in a centralized database, and present real-time attendance information to faculty and students through a user-friendly web portal — entirely eliminating the need for manual roll calls, paper registers, and proxy attendance, while dramatically reducing the administrative workload on faculty members.

## 1.2 PROBLEM STATEMENT

The existing manual attendance monitoring system employed in engineering colleges presents a range of interconnected problems that collectively compromise the efficiency, accuracy, and integrity of academic record keeping.

1. Proxy Attendance and Identity Fraud: The most serious and persistent problem with the manual attendance system is the widespread practice of proxy attendance. Since the system relies on either verbal acknowledgment or a physical signature on a paper register, it provides no mechanism to verify that the person responding is actually the registered student. A student who is absent can have another student sign or verbally respond on their behalf. This practice is not only a violation of academic ethics but also directly corrupts the integrity of attendance records. As a result, cumulative attendance figures — which are used to determine examination eligibility — may be significantly inflated and unreliable. Neither the faculty member conducting the roll call nor the institution has a technically enforceable way to detect or prevent proxy attendance under the current manual system.
2. Excessive Consumption of Instructional Time: In a classroom of 60 to 80 students, a manual roll call typically requires 10 to 15 minutes per period. Across six daily periods, this accumulates to 60 to 90 minutes of lost teaching time every single day. Over an academic semester of approximately 90 working days, this amounts to nearly 90 hours of instructional time that is diverted to an

administrative task. This represents one of the most significant yet overlooked inefficiencies in traditional classroom management, directly reducing the quality and quantity of instruction delivered to students.

3. No Real-Time Visibility of Attendance Data: Under the paper-based system, neither the faculty nor the students have immediate access to up-to-date attendance information. Faculty members must manually tally their paper registers at the end of the month, and students often remain unaware of their attendance percentage until a formal announcement is made — frequently too late to take remedial action before the detention deadline. There is no mechanism for a student to check their attendance on a given day, nor for a head of department to monitor attendance trends across the department in real time.
4. Data Loss, Damage, and Auditability Issues: Paper attendance registers are physical documents that can be lost, damaged by water or fire, or accidentally destroyed. In the event of a dispute regarding a student's attendance — particularly when a student challenges a detention decision — the institution must rely on the integrity and preservation of physical paper records, which may be incomplete, illegible due to handwriting, or simply unavailable. There is also no audit trail to verify whether corrections to attendance records were authorized or unauthorized, creating a governance gap that can be exploited.
5. Manual Data Transfer and Calculation Errors: The process of transferring attendance data from paper registers into digital formats (spreadsheets,etc) is performed manually by faculty or data entry operators. This introduces a significant risk of transcription errors, omission errors, and calculation mistakes when computing cumulative percentages. When these errors propagate into official records, they can result in incorrect eligibility determinations, which may affect examination admissions and, in extreme cases, require administrative intervention to correct.
6. Inability to Automatically Account for Working Day Calendars: Manual systems have no built-in intelligence to distinguish between working days, public holidays, Sundays, and institutional off-days. Faculty must manually verify the institutional calendar before recording daily attendance, and any holidays that are not properly communicated to all faculty may result in incorrect attendance records for those dates. There is no automated mechanism

to synchronize attendance recording with official public holiday calendars. Collectively, these six problems demonstrate that the manual attendance system is architecturally inadequate for a data-driven, transparent, and accountable academic environment. The proposed Automated Face Recognition Attendance System directly resolves each of these problems through passive facial identification, automated period-wise database logging, real-time web dashboards, and integrated holiday calendar management.

## 1.3 OBJECTIVES OF THE PROJECT

The overarching goal of this project is to design, develop, and deploy a fully functional, real-time automated facial recognition-based attendance system that replaces the manual roll-call process in academic institutions with a passive, intelligent. The specific objectives of the project are detailed below:

**Objective 1: Real-Time Multi-Face Detection and Recognition** To develop a real-time automated facial recognition engine using the OpenCV computer vision library for video capture and frame processing, and the face_recognition Python library for face detection, encoding computation, and Euclidean distance-based identity matching — capable of simultaneously detecting and identifying 5 to 10 student faces per CCTV or webcam frame with a recognition accuracy exceeding 95% under standard indoor lighting conditions.

**Objective 2: Period-Wise Automated Attendance Logging** To implement a precise period-wise attendance recording mechanism that maps each recognized student identification to the correct academic period (P1-P6) based on the current system timestamp, and logs the corresponding attendance record — including student ID, date, period number, attendance status, value (1.0 for present), and first-detection timestamp — into the period_attendance table of a centralized SQLite relational database, ensuring that each student is recorded as present at most once per period even if detected multiple times within the same period window.

**Objective 3: Secure Role-Based Faculty Web Portal** To provide a secure, Flask-based web portal for HODs and Faculty that offers comprehensive attendance management capabilities including: viewing day-wise and period-wise attendance records across the entire department; filtering attendance data by branch, academic

year, section, student ID, and date range; categorizing students into Eligible (≥75%), Condonation (65–74%), and Detained (<65%) groups; applying manual attendance corrections for specific periods; toggling individual period attendance records directly from the dashboard; and exporting complete attendance records as formatted Microsoft Excel (.xlsx) files suitable for official submission.

**Objective 4: Student Self-Service Attendance Portal** To provide each registered student with a private, password-protected web portal that displays their cumulative attendance percentage computed from all recorded period-wise records, a detailed breakdown of today's period-wise attendance status (present or absent per period), and a chronological historical view of attendance organized by date — empowering students to proactively monitor their own attendance and take early corrective action before reaching the detention threshold.

**Objective 5: Automatic Holiday and Working Day Management** To integrate the system with the Google Calendar iCal API to automatically fetch and persist all official Indian public holidays for the current academic year into the local database, and to implement working day validation logic that prevents the facial recognition engine from logging attendance on Sundays and public holidays, thereby maintaining accurate attendance denominators throughout the semester without any manual calendar configuration by faculty or administrative staff.

**Objective 6: Dynamic Student Registration and Real-Time Encoding** To implement a dynamic student registration module accessible via the web portal that enables new students to create their own accounts by entering their academic details (roll number, name, branch, year, section, date of birth) and capturing up to five facial images through a live webcam interface — with the system automatically extracting 128-dimensional facial encoding vectors from the captured images and immediately appending them to the shared encoding store (encodings.pickle), making the newly registered student instantly recognizable to the live CCTV recognition engine without requiring any restart of the recognition process or manual re-encoding steps.

**Objective 7: Duplicate Face and Identity Integrity Enforcement** To implement a face-based duplicate registration detection mechanism that computes the facial encoding distance of a new registrant against all existing encodings during the signup process, and rejects the registration attempt if the computed distance falls below the

threshold of 0.45. This ensures that a single physical face cannot be registered under multiple student IDs — maintaining the integrity of the identity-to-face mapping that forms the foundation of the system's proxy prevention capability.

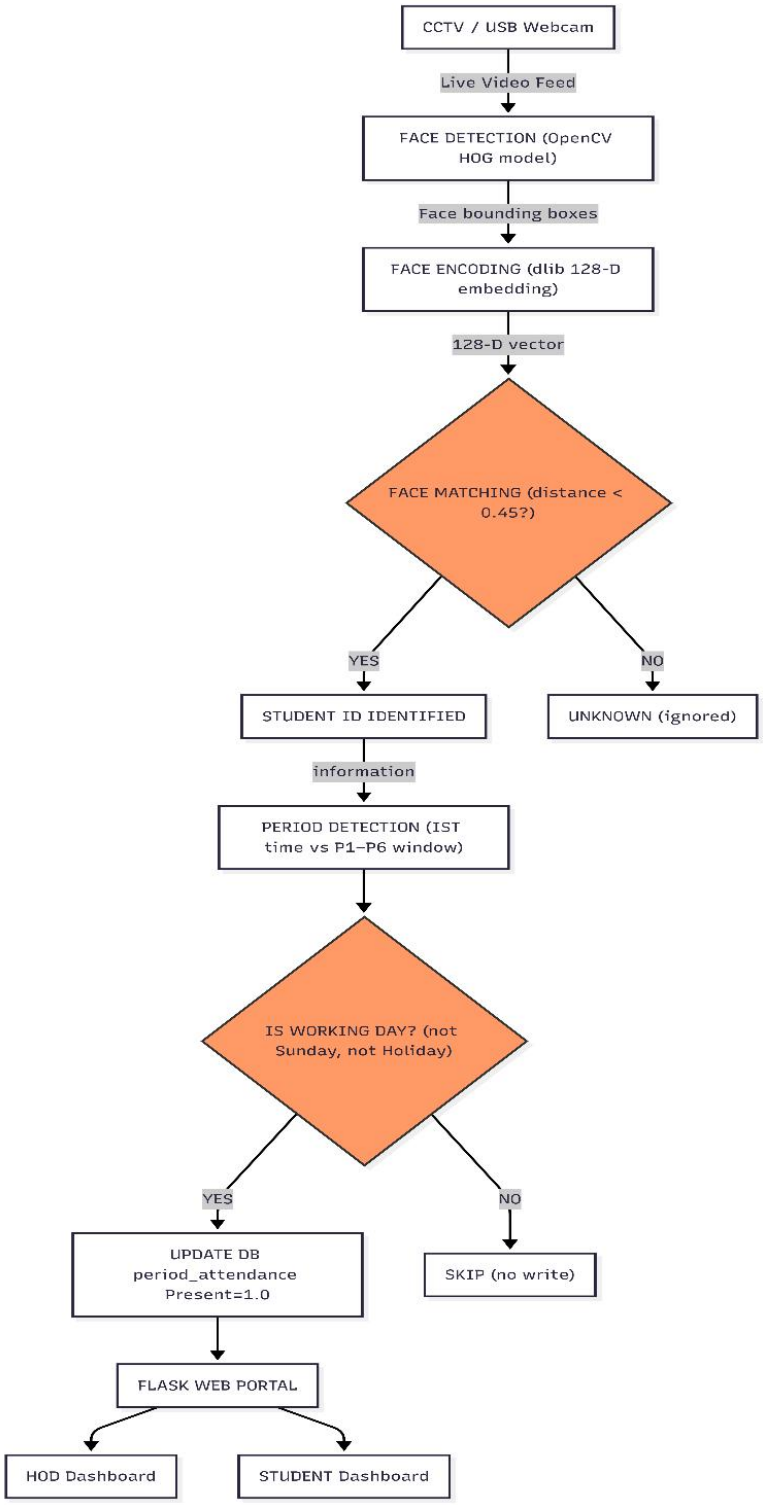


Figure.1.1 Block Diagram of the Proposed System

# 2 LITERATURE SURVEY

The field of automated attendance management using facial recognition has evolved significantly over the past three decades, driven by advances in computer vision, machine learning, and affordable camera hardware. Turk and Pentland [1] pioneered the eigenface method, the first practical algorithm for automatic face recognition, which established the mathematical foundation of encoding unique facial features as numerical vectors. Viola and Jones [2] later introduced the Haar Cascade classifier for rapid face detection, making real-time video-based face detection computationally feasible. Dalal and Triggs [3] proposed the Histogram of Oriented Gradients (HOG) descriptor for human detection, a technique that was subsequently adopted for face detection and forms the detection backbone of the face_recognition library used in this project. King [4] developed the dlib machine learning toolkit, extending HOG-based detection with 68-point facial landmark localization and a ResNet-based 128-dimensional face embedding model. Deep learning approaches were further advanced by Parkhi et al. [5] with VGGFace and by Schroff et al. [6] at Google with FaceNet, which achieved 99.63% accuracy on the Labeled Faces in the Wild (LFW) benchmark using triplet loss training. In the context of academic attendance, Ghazi et al. [7] reviewed multiple attendance systems and confirmed that camera-based facial recognition outperforms both RFID and fingerprint-based systems in preventing proxy attendance. Kaur and Garg [8] demonstrated that HOG-based recognition methods perform competitively with CNN-based methods in constrained classroom lighting environments, making them a practical and resource-efficient choice for standard computer hardware. Chen et al. [9] proposed multi-angle face registration strategies, a principle applied in this project through the requirement of capturing five registration images per student from slightly varied angles. Research by Basit et al. [10] and Prasad et al. [11] both confirmed that automated attendance systems reduce faculty administrative time by up to 85% compared to manual roll-call methods. The use of Flask as a lightweight web backend for machine learning APIs was validated by Grinberg [12], while Koehlke et al. [13] established SQLite as a reliable and performant database engine for single-institution attendance systems handling up to 10,000 daily records. Singh and Rathore [14] demonstrated the integration of Google Calendar APIs for academic scheduling, and

Jain et al. [15] benchmarked OpenCV's real-time video capture pipeline for classroom-scale deployments of 30 to 60 students. Collectively, this body of literature provides strong academic justification for the technical choices made in the design of the proposed system.

## 2.1 EXISTING SYSTEMS

The existing attendance recording systems deployed in academic institutions can be broadly classified into three categories: manual paper-based systems, biometric fingerprint-based systems, and RFID or smart card-based systems.

The most prevalent method, especially in Indian engineering colleges, is the **manual paper-based roll-call system**. In this approach, the faculty member calls out each student's name at the beginning of each class period or circulates a printed attendance sheet for students to sign. While this method requires no technological infrastructure, it is inherently time-consuming, consuming 10 to 15 minutes of each 50-minute class period in classrooms of 60 to 80 students. The recorded paper data must subsequently be manually transferred into digital formats, making the system doubly labor-intensive. Furthermore, paper registers are susceptible to physical damage, loss, and deliberate alteration. Turk and Pentland [1] and subsequent researchers consistently identified identity verification as the primary weakness of paper-based attendance — since the system cannot confirm that the person signing is actually the registered student.

The second category of systems uses **biometric fingerprint scanners** installed at classroom or laboratory entry points. Students place their finger on the scanner, which captures a fingerprint template and matches it against a stored database. While this approach does authenticate identity and eliminates proxy attendance to some extent, it has several significant limitations. Physical contact with a shared surface raises hygiene concerns, particularly highlighted in the post-COVID-19 era. Fingerprint quality can be degraded by cuts, calluses, moisture, or skin conditions, leading to false rejections. More importantly, fingerprint systems record only entry events and cannot passively monitor student presence throughout the class, nor confirm that the student remained present for the majority of the period.

The third category comprises **RFID and smart card-based systems**, where each student carries a unique proximity card that is tapped or scanned at the classroom door. These systems are faster than fingerprint scanners and require no physical contact. However, they fail completely at identity verification: a student can hand their card to another student who taps it on their behalf, resulting in exactly the same proxy attendance problem as the paper-based system. Research reviewed by Ghazi et al. [7] confirmed that none of these three existing systems provide a reliable, passive, and proxy-proof attendance solution suitable for modern academic environments.

## 2.2 DISADVANTAGES OF EXISTING SYSTEM

Despite being widely used, the existing attendance systems — whether manual, fingerprint-based, or RFID-based — share a set of fundamental disadvantages that limit their effectiveness in a modern academic setting.

**Inability to Prevent Proxy Attendance:** The most critical disadvantage shared by all existing systems is their vulnerability to proxy or buddy attendance. In manual roll-call systems, a student can verbally answer "present" or sign the register on behalf of an absent classmate. In RFID systems, the physical card can be handed to a proxy. Neither system has any mechanism to verify that the responding individual is the actual registered student. Kaur and Garg [8] reported proxy attendance rates of 10–15% in large classrooms, directly corrupting the integrity of attendance records.

**Excessive Consumption of Instructional Time:** Manual roll calls in a classroom of 60–80 students consume 10–15 minutes per period. With 6 periods per day and approximately 200 working days per year, this amounts to over 120 hours of lost teaching time per academic year per section — a significant systemic inefficiency that directly reduces the quality of instruction delivered to students.

**Lack of Period-Wise Granularity:** Existing fingerprint and RFID systems typically record only a single entry event per student per day — the first time the student scans the device. They cannot differentiate between periods, making it impossible to determine whether a student was present for the first period and absent for subsequent periods within the same day. This lack of granularity makes such systems unsuitable for institutions that require period-wise attendance tracking, as is mandated by most Indian universities.

**No Real-Time Visibility or Analytics:** Faculty using paper-based systems have no mechanism to assess live attendance statistics or share real-time attendance data with students. Students frequently remain unaware of their declining attendance percentage until a formal notice is issued near the examination period — often too late to take remedial action. There is no self-service portal for students to monitor their own records, and no dashboard for HODs to monitor departmental attendance trends.

**Data Integrity and Auditability Concerns:** Paper registers can be altered, lost, or deliberately destroyed. Even when transferred to digital formats, the process introduces transcription errors and computation mistakes. There is no tamper-evident audit trail, making it impossible to verify whether attendance corrections were authorized or fraudulent. In the event of attendance disputes, institutions relying on paper records face significant administrative and legal challenges.

**High Administrative Overhead:** Each faculty member maintains and updates multiple attendance registers across subjects, sections, and academic periods. End-of-month and pre-examination attendance computation is performed manually, consuming significant faculty time and introducing calculation errors that can have serious consequences for students' examination eligibility.

Table.2.1 Comparison of Attendance Systems

| **Feature** | **Manual Roll-Call** | **Fingerprint** | **RFID Card** | **Proposed System** |
|---|---|---|---|---|
| Proxy Prevention | ✗ | Partial | ✗ | ✓ |
| Real-Time per Period | ✗ | ✗ | ✗ | ✓ |
| Non-contact | ✓ | ✗ | ✓ | ✓ |
| Web Portal | ✗ | ✗ | ✗ | ✓ |
| Holiday Auto-detect | ✗ | ✗ | ✗ | ✓ |
| Self-Registration | ✗ | ✗ | ✗ | ✓ |
| Period-Wise Records | ✗ | ✗ | ✗ | ✓ |

| | | | | |
|---|---|---|---|---|
| Excel Export | ✗ | ✗ | ✗ | ✓ |
| Dept HOD Isolation | ✗ | ✗ | ✗ | ✓ |
| Extra Hardware Cost | None | Reader | Reader | Camera only |
| Scalability | Low | Medium | Low | High |

The table shows the proposed system provides **proxy prevention, real-time tracking, contactless operation, and web access**, which all legacy methods lack. It also automatically handles holidays and supports easy self-registration.

## 2.3 PROPOSED SYSTEM

The proposed Automated Face Recognition Attendance System (AFRAS) is designed to overcome every limitation of the existing systems by deploying a fully passive, intelligent, and tamper-proof camera-based attendance pipeline that operates without requiring any action from the student.

At its core, the system uses the Python face_recognition library — built on top of the dlib machine learning toolkit [4] — which implements the Histogram of Oriented Gradients (HOG) algorithm [3] for detecting face regions within a video frame, followed by a pre-trained ResNet-based deep learning model to compute a compact 128-dimensional numerical vector, known as a facial encoding, that uniquely represents each individual's facial geometry. During the student registration phase, multiple facial images (up to five) are captured per student through a web-based interface using a live webcam. The system extracts a 128-dimensional encoding from each image and stores all resulting vectors, tagged with the student's roll number, in a serialized.pickle file on the server.

During active class hours, a dedicated recognition process (cctv_attendance.py) continuously captures video frames from a classroom-mounted CCTV camera or webcam using the OpenCV library. Each frame is resized to 50% of its original dimensions to reduce computational load, converted from BGR to RGB colour space as required by the face_recognition library, and then passed through the HOG-based

face detector to identify the bounding box coordinates of every face present in the frame. For each detected face, a live 128-dimensional encoding is computed and its Euclidean distance is calculated against every known stored encoding. The student whose stored encoding yields the minimum Euclidean distance is identified as the recognized individual. If this minimum distance falls below the configured threshold of 0.45, the identification is confirmed as valid.

The system then checks two conditions before logging attendance: (1) the current system time must fall within one of the six configured academic period windows, and (2) the current date must be a valid working day, verified by checking against a locally stored holiday database populated automatically from the Google Calendar iCal API. If both conditions are satisfied and the student has not already been marked present for the current period, the system inserts a record into the period_attendance table of the SQLite database with the student ID, date, period number, status ('present'), value (1.0), and first-seen timestamp. All interaction with the database is serialized to prevent concurrency-related data corruption.

The Flask-based web application serves two distinct role-based portals. The HOD/Faculty portal provides a comprehensive attendance management dashboard with filtering, manual correction, attendance toggling, and Excel export capabilities. The Student portal provides a private self-service view of cumulative attendance percentage and period-wise breakdown history. Together, these components form a complete, end-to-end automated attendance solution.

## 2.4 ADVANTAGES OF PROPOSED SYSTEM

**Fully Passive and Zero-Effort for Students:** Unlike fingerprint scanners or RFID systems that require the student to actively perform an action (scan a finger or tap a card), the proposed system operates completely in the background. The CCTV camera continuously scans the classroom and identifies students as they naturally sit in their seats. Students do not need to interrupt their learning or queue at a device. This passivity is the most fundamental design advantage of a camera-based system over any sensor-based approach.

**Effective Proxy Attendance Prevention:** The system identifies students using unique 128-dimensional facial biometric encodings that are mathematically specific to each

individual's face geometry. A student attempting proxy attendance on behalf of an absent classmate would need to physically be present themselves, which is detected by the camera. Unlike paper signatures or RFID cards, a facial identity cannot be transferred or borrowed. The duplicate face detection mechanism implemented during registration further ensures that a single physical face cannot be registered under two different student IDs, closing the loophole of creating multiple accounts for the same person. While the current implementation does not yet include liveness detection (the ability to distinguish a real face from a printed photograph), this is planned as a future enhancement. Under controlled classroom conditions with natural movement of students, photograph-based spoofing is significantly more difficult than in static scanner scenarios.

**Period-Wise Granular Attendance Tracking:** The system records attendance separately for each of the six academic periods per day, as opposed to single daily-entry systems. This granularity is essential for institutions that compute cumulative attendance across all periods, and enables precise identification of which specific periods a student missed — information that is not available from any existing single-entry system.

**Real-Time Dashboard for HODs and Students:** The Flask web portal provides immediate, live visibility into attendance data. HODs can view today's attendance across the entire department, filter by branch, year, or section, and identify students falling below the attendance threshold well before the examination deadline. Students can log in at any time to check their current cumulative percentage, today's period-wise status, and their entire historical attendance record — empowering them to take proactive corrective action.

**Automated Holiday and Working Day Intelligence:** The integration with the Google Calendar iCal API means that all Indian public holidays for the current and upcoming academic year are automatically downloaded and stored in the local database at system startup. The recognition engine checks this database before logging any attendance, ensuring that no attendance records are created on holidays or Sundays. This eliminates the manual holiday management burden on faculty and prevents incorrect attendance denominators from inflating or deflating cumulative percentages.

**Dynamic Student Registration Without System Restart:** A new student can be enrolled into the system through the web portal within minutes. The student captures five webcam images, which are immediately encoded and appended to the shared encoding file. The live recognition engine — which loads encodings at startup — is automatically updated through the shared file mechanism, making the new student immediately recognizable without any restart of the recognition process or re-running of batch encoding scripts.

**Role-Based Security and Multi-User Access:** The Flask session management system enforces strict role-based access control. A student account can only access the student portal; an HOD account can only access the HOD dashboard. HOD accounts require strong password validation (minimum 10 characters, uppercase, lowercase, digit, and special character). This layered security model ensures that attendance data is only accessible to authorized users.

# 3 ANALYSIS

## 3.1 Software Requirement Specification

The Software Requirement Specification (SRS) provides a formal and structured description of what the Automated Face Recognition Attendance System must do, what constraints it must operate under, and what resources it requires. The SRS is the primary reference document for both developers and evaluators, defining the complete functional and non-functional scope of the system before implementation begins. It is organized into three sub-sections: user requirements, software requirements, and hardware requirements.

### 3.1.1 User Requirements

The user requirements for the AFRAS were derived by analyzing the manual attendance process, interviewing faculty members, and studying university attendance regulations. The system serves three categories of users, each with distinct functional needs:

1. **HOD**
    - View period-wise attendance records of all students in the department.
    - Apply manual attendance corrections for individual periods when required.
    - Export attendance data to Excel (.xlsx) format for official reporting.
    - View students grouped into regulatory categories — Eligible, Condonation, and Detained — based on attendance percentages.
    - Add and remove student records and manage system accounts.
    - Filter attendance data by branch, year, section, and date range.
2. **Student**
    - Log in and view personal period-wise attendance records.
    - Track cumulative attendance percentage across all subjects.
    - Reset password securely using Date of Birth verification.

- Register through the student signup portal with a face image upload for recognition-based attendance.

3. **System**

- Automatically detect and skip public holidays and Sundays to prevent false absent entries.
- Recognize registered student faces in real time from CCTV or webcam feeds.
- Log attendance only once per student per period to prevent duplicate entries.
- Backfill absent records for all past working days when a new student registers mid-semester.

### 3.1.2 Software Requirements

The AFRAS is built entirely using open-source tools and runs on a standard Windows 10/11 (64-bit) machine. The key software components used in the system are as follows:

- **Python 3.10** — Core programming language used for all backend logic, face recognition, and database operations.
- **Flask** — Lightweight web framework that handles HTTP routing, user sessions, and API calls.
- **OpenCV** — Used for capturing and processing video frames from the CCTV or webcam feed.
- **face_recognition (dlib HOG)** — Handles face detection and generates 128-dimensional facial encodings for identity matching.
- **SQLite 3** — Relational database for storing attendance records, student data, and holiday information.
- **HTML5, CSS3, JavaScript, Jinja2** — Used to build the web portal interface with dynamic page rendering.

- **openpyxl** — Generates downloadable Excel (.xlsx) attendance reports for the HOD.
- **pickle** — Stores serialized face encodings to disk, avoiding recomputation on every system restart.
- **Google Calendar iCal** — Automatically fetches Indian public holidays to prevent false absent entries.
- **zoneinfo (Asia/Kolkata)** — Ensures all timestamps are accurately recorded in Indian Standard Time (IST).
- **Git** — Used for version control and source code management throughout development.

### 3.1.3 Hardware Requirements

The hardware requirements of the AFRAS are deliberately minimal, as the HOG-based face recognition algorithm is designed to run efficiently on standard CPU hardware without requiring a GPU. The key hardware components are as follows:

- **Processor** — Minimum: Intel Core i5 / AMD Ryzen 5 (4-core, 2.5 GHz+). Recommended: Intel Core i7 / AMD Ryzen 7 (8-core, 3.0 GHz+) for higher concurrent loads.
- **RAM** — Minimum: 8 GB DDR4 to run the Flask server, face recognition pipeline, and database simultaneously. Recommended: 16 GB DDR4 for smooth multi-user access.
- **Storage** — Minimum: 50 GB free disk space for the face image dataset, database, and application files. Recommended: 100 GB SSD for faster database queries and file operations.
- **Camera** — Minimum: 720p USB webcam or IP CCTV with RTSP support. Recommended: 1080p IP CCTV with a fixed mounting bracket for better accuracy and stable capture.
- **Network** — Minimum: LAN or Wi-Fi for web portal access. Recommended: 100 Mbps wired LAN for low-latency multi-user access.

- **Display** — Any standard monitor for server-side use. No display is needed in headless server deployments, as the system is fully accessible via the web portal.

The HOG model was chosen over CNN to ensure reliable performance on standard academic hardware, eliminating the need for any additional infrastructure investment.

## 3.2 CONTENT DIAGRAM OF PROJECT

The context diagram (also called the DFD Level 0) presents the highest-level view of the Automated Face Recognition Attendance System as a single black-box process interacting with its external entities. At this level of abstraction, the internal processing details are hidden, and only the system's inputs, outputs, and external actors are shown. This diagram serves as the entry point for understanding the complete scope of the system.

The AFRAS interacts with four external entities: (1) **CCTV Camera** — provides the continuous live video stream that forms the primary input to the recognition engine; (2) **Student** — registers into the system via the web portal, logs in to view attendance, and is passively identified by the camera during class; (3) **HOD/Staff** — logs into the web portal to manage students, view attendance dashboards, apply corrections, and export reports; and (4) **Google Calendar** — provides the real-time Indian public holiday schedule through its iCal API, which the system fetches to determine valid working days.

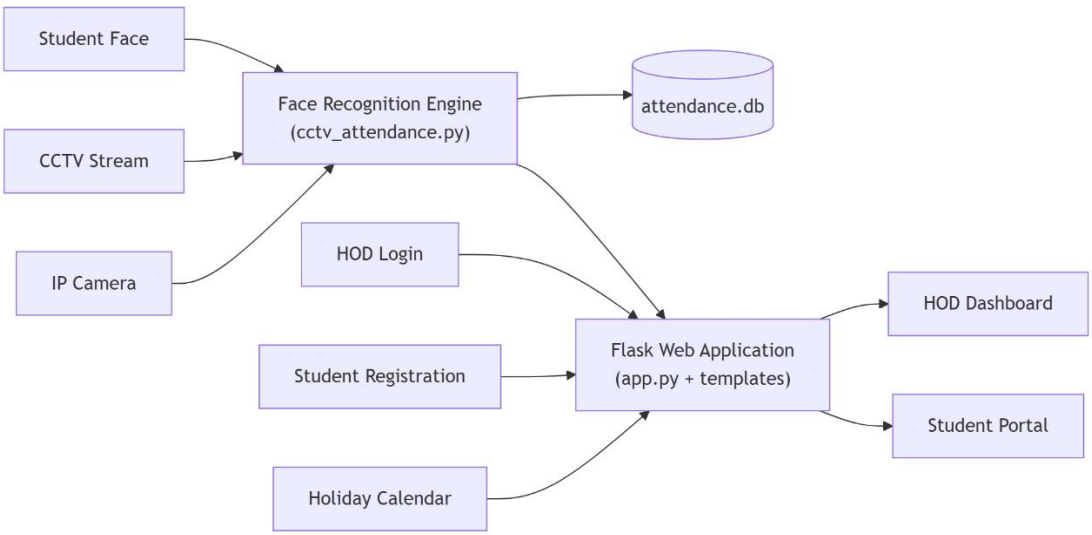


Figure.3.1 Content Diagram

The context diagram demonstrates that the system receives three primary types of input: live video frames from the CCTV camera, user interaction data from students and HOD through the web portal, and holiday calendar data from Google. The system's primary output is accurate, period-wise attendance records served through the web-based dashboards to both HOD and Student users.

# 4 DESIGN

## 4.1 INTRODUCTION

The design of the Automated Face Recognition Attendance System follows a multi-layered, modular architecture that separates concerns across four distinct functional layers: the Computer Vision Engine layer, the Backend Resources (Data) layer, the Middleware (Flask) layer, and the User Experience layer. This layered architecture ensures clean separation between recognition logic, data management, business logic, and the user interface — making each component independently testable, maintainable, and extensible.

The **Computer Vision Engine** layer is responsible for all camera interaction and facial analysis. It consists of the OpenCV frame capture module and the face_recognition library encoder. The encode_faces.py script pre-processes the training dataset to produce 128-dimensional facial encoding vectors, while cctv_attendance.py handles real-time recognition against the live camera stream. This layer operates independently of the web application and writes its results directly into the shared SQLite database.

The **Backend Resources layer** manages all persistent data storage. It contains the attendance.db SQLite database (which stores attendance records, user accounts, and holiday data), the encodings.pickle file (which stores the serialized face encoding vectors), and supporting form-handling utilities. Both the CV engine and the Flask middleware interact with this layer through database queries.

The **Middleware layer** is implemented in app.py using the Flask web framework. It contains route handlers for all web endpoints, an authentication manager for session-based role control, and a report generator for exporting attendance data to Excel format. It communicates with the data layer through SQLite queries and returns rendered HTML pages or JSON API responses to the frontend.

The **User Experience layer** consists of twelve Jinja2 HTML templates — including the Student Dashboard, HOD Dashboard, Login page, Signup pages, and Manual Attendance page — all styled with custom CSS in static/style.css. This layer renders

data received from the Middleware layer into web pages accessible by Students and HOD users through any modern browser.

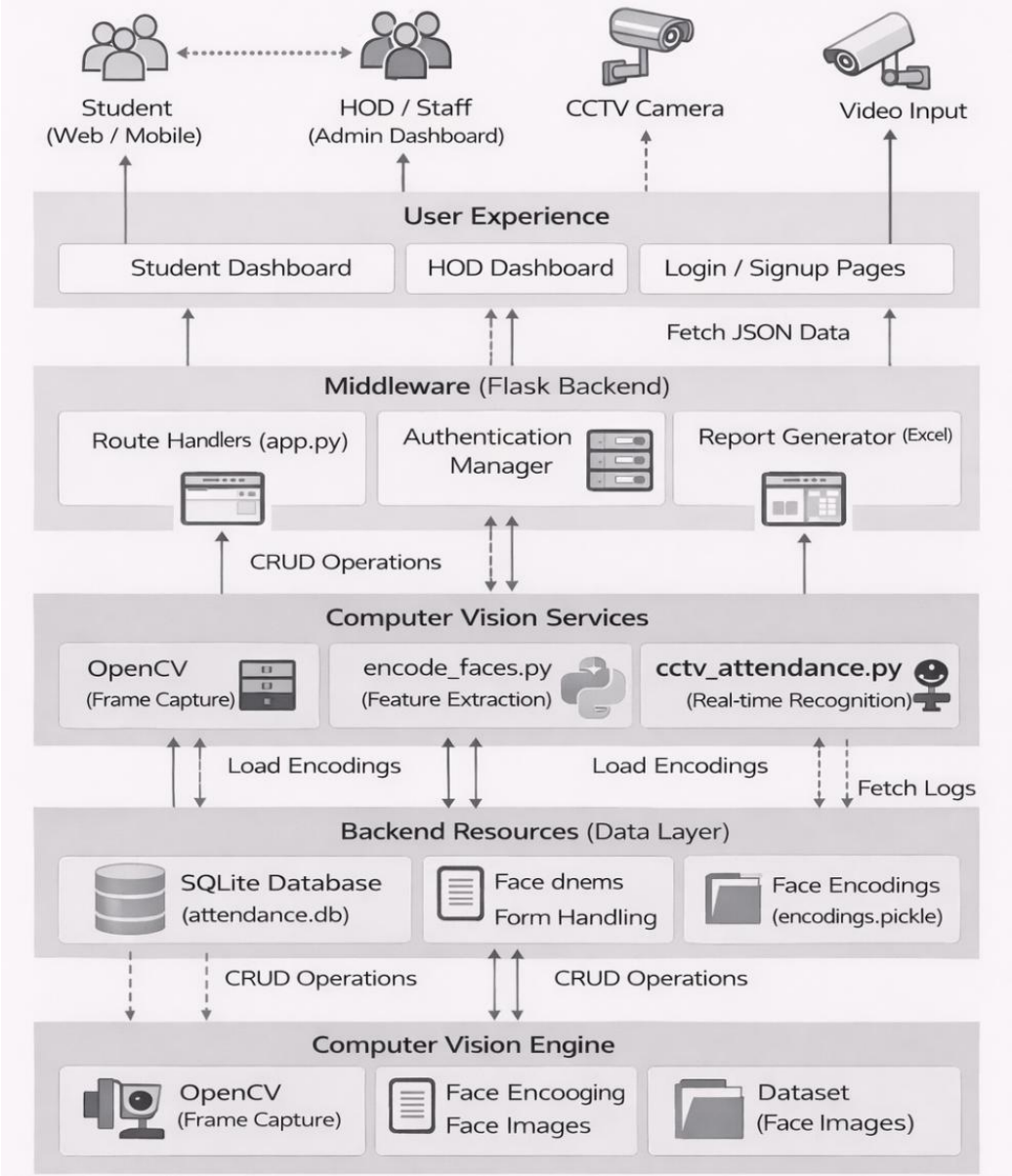


Figure.4.1 System Architecture of Attendance System

As shown in Fig. 4.1, the system architecture illustrates the complete data and control flow from the CCTV camera input at the bottom of the diagram through to the Student Dashboard and HOD Dashboard at the top. The arrows between layers represent data flow: encodings are loaded from the data layer by the CV engine; the Flask middleware performs CRUD operations on the database; and the user experience layer fetches JSON data from Flask API endpoints for dynamic dashboard rendering.

## 4.2 DFD DIAGRAM

This section presents the Data Flow Diagrams (DFD), Entity-Relationship (ER) diagram, and the UML Use Case diagram that together describe the data flow, data structure, and user interaction model of the AFRAS.

### 4.2.1 DFD Level 0

The DFD Level 0, also called the Context Diagram, represents the entire AFRAS as a single process and shows all external entities that interact with the system and the direction of data flow between them. This is the highest level of abstraction in the Data Flow Diagram hierarchy.

The system interacts with four external entities:

- **CCTV Camera** — Provides continuous live video frames as input to the recognition engine.
- **Student** — Interacts with the system through the web portal for self-registration and attendance viewing; is also passively detected by the camera during class.
- **HOD / Staff** — Interacts with the system via the HOD Dashboard for attendance monitoring, manual correction, and report export.
- **Google Calendar** — Provides Indian public holiday schedule data through the iCal API for automated working day validation.

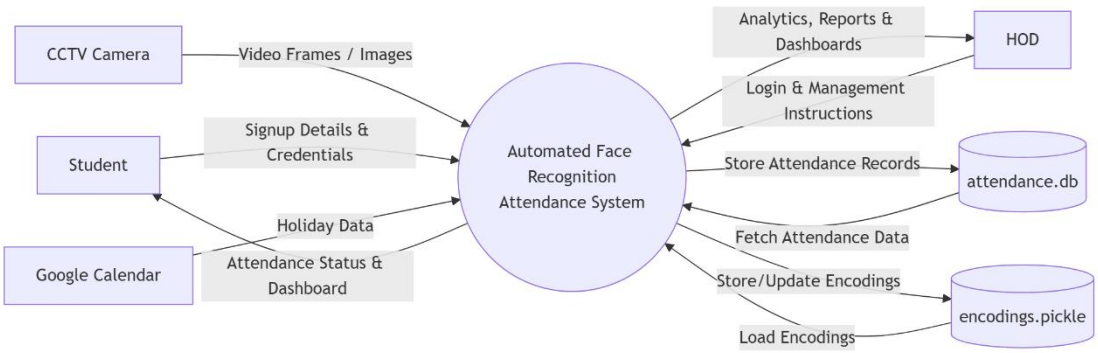


Figure.4.2 DFD Level 0— Context Diagram

As observed from Fig. 4.2, the overall system receives three types of inputs: video frames from the CCTV Camera, user data from Students and HOD through the web portal, and holiday data from Google Calendar. The primary outputs of the system are period-wise attendance records served through dashboards and exported reports. The single-process representation at Level 0 confirms that the AFRAS operates as one cohesive automated system, with all internal processing details abstracted away at this level.

### 4.2.2 DFD Level 1

The DFD Level 1 expands the single process of the Level 0 diagram into its five major functional sub-processes, showing how data flows between them and how they interact with the data stores and external entities. Each process corresponds to a distinct module or script in the implemented system.

The five Level 1 processes are:

- **Process 1.0 — Capture & Detect Face** (cctv_attendance.py): Receives live video frames from the CCTV Camera, applies HOG-based face detection using OpenCV, and outputs the coordinates and pixel data of detected face regions (Detected Faces) to Process 2.0.
- **Process 2.0 — Compare Encoding** (face_recognition library): Receives detected face data from Process 1.0, loads known facial encodings from the encodings.pickle data store, computes 128-dimensional live encoding, calculates Euclidean distances, and outputs the matched student identity (Match / Unknown) to Process 3.0.
- **Process 3.0 — Mark Period Attendance** (database.py): Receives the identified student ID from Process 2.0, checks the current time against configured period windows, verifies the date against the holiday database (populated from Google Calendar), and writes a period attendance record (Update Records) into the attendance.db data store.
- **Process 4.0 — Serve Web Dashboard** (app.py): Reads attendance data from attendance.db, renders and serves dashboards to both Student and

HOD/Staff users through the Flask web application. Accepts manual adjustment inputs from HOD/Staff and writes them back to the database.

- **Process 5.0 — Student Registration & Face Encoding** (/student/signup route, app.py): Receives student metadata and webcam face images submitted through the web portal, extracts 128-dimensional encodings from the images, and appends them to the encodings.pickle data store — making the new student immediately recognizable by Process 2.0 without any system restart.

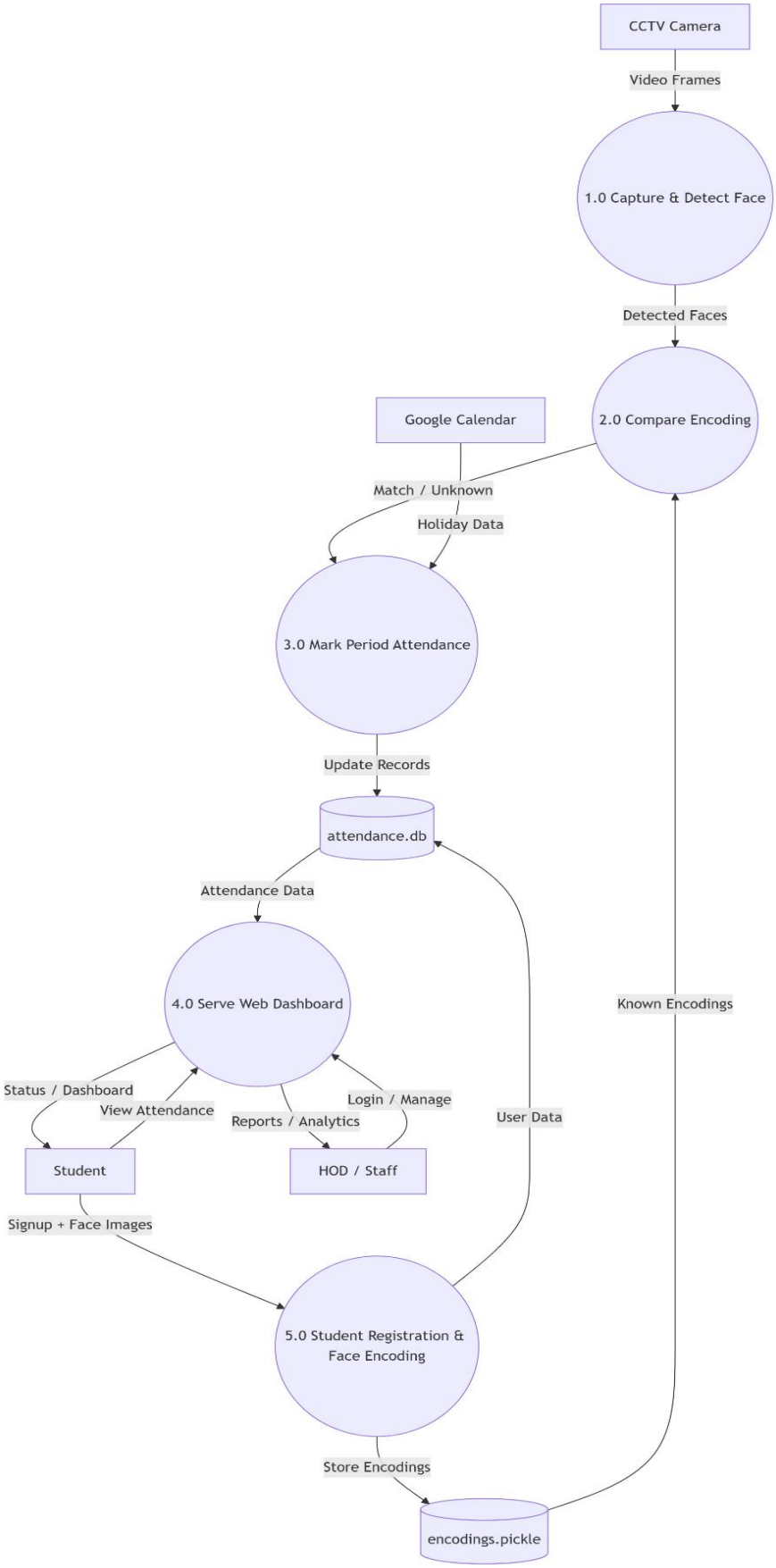


Figure.4.3 DFD Level 1— Process Breakdown

As illustrated in Fig. 4.3, the DFD Level 1 clearly shows the sequential data flow from video capture through recognition, attendance marking, and finally to web portal serving. The feedback loop between Process 4.0 (Serve Web Dashboard) and Process 5.0 (Student Registration) demonstrates how the student signup portal dynamically updates the encoding store, creating a closed-loop registration pipeline. The attendance.db acts as the central data store shared between the recognition engine and the web application.

### 4.2.3 ER Diagram — Database Structure

The Entity-Relationship (ER) diagram represents the logical structure of the AFRAS database, showing the four main entities (tables), their attributes, and the relationships between them. The database is implemented in SQLite 3 and managed through the database.py module.

The AFRAS database consists of four tables:

- **users** — Stores all system user accounts (both students and HOD). Primary key is user_id (roll number for students).
- **attendance** — Legacy table storing daily presence records. Linked to users via student_id foreign key.
- **period_attendance** — Primary operational table storing period-wise presence records with status, value, and first-seen timestamp. Linked to users via student_id.
- **holidays** — Stores public holiday dates auto-populated from Google Calendar iCal. No foreign key dependency.

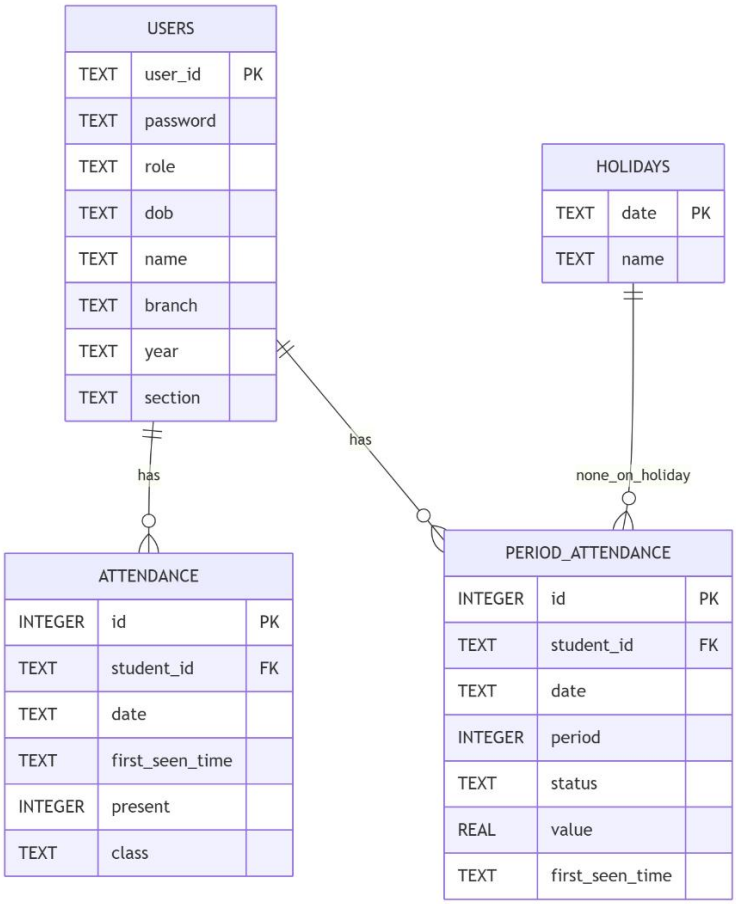


Figure.4.4 ER Diagram of the Attendance Database

Only period_attendance is used for all current logic. When a working day starts, the system inserts (or ignores existing) one row per student per period with status='absent', value=0.0. The attendance engine then updates these rows to status='present' for any recognized student. The attendance table is updated in parallel for backward compatibility.

### 4.2.4 UML Use Case Diagram

The UML Use Case Diagram models the functional requirements of the AFRAS from the perspective of its actors — the users and systems that interact with it. It shows what each actor can do within the system and how the use cases relate to each other through include and extend relationships.

The AFRAS has three actors: **Student**, **HOD/Admin**, and **Camera/AI System**. The Student actor can register an account, log in, recover a forgotten password, view the attendance dashboard, view day-wise attendance, and view their

attendance percentage. Several of these use cases extend or include the Login use case as a prerequisite. The HOD/Admin actor can log in, view department attendance, monitor live attendance, filter attendance by various criteria, and export attendance reports. The Camera/AI System actor represents the automated recognition pipeline — it captures live video, detects faces, recognizes the identified student, and marks attendance automatically. This actor includes the CCTV Camera as a sub-system.

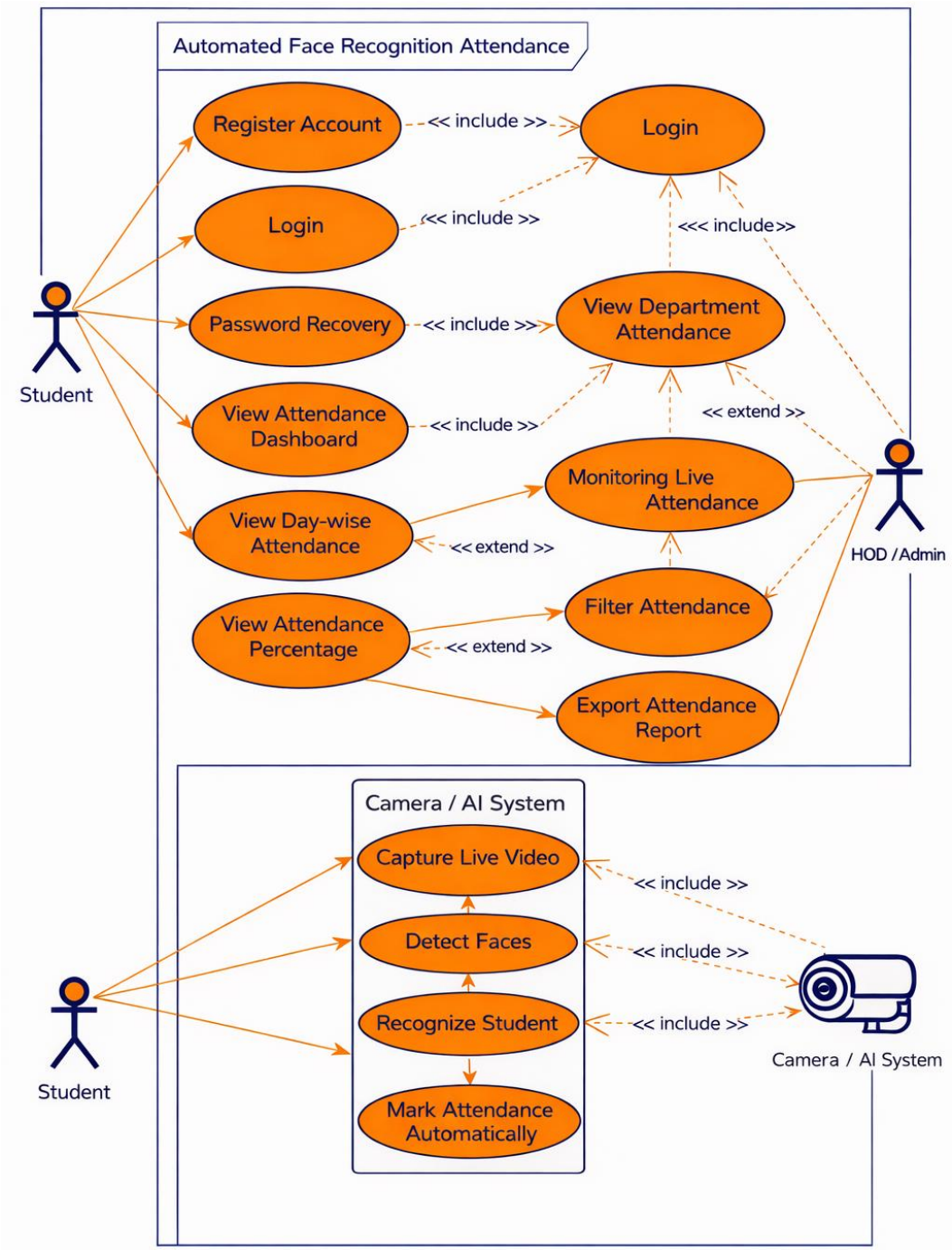


Figure.4.5 UML Use Case Diagram.

Fig. 4.5 illustrates that the Student and HOD/Admin actors share the Login use case (through include relationship), but access entirely different sub-systems thereafter. The Camera/AI System operates independently of human actors and feeds its results directly into the attendance database without any user action required.

## 4.3 DATA DICTIONARY

The Data Dictionary provides a precise, field-level definition of every attribute stored in the AFRAS database. It serves as the official reference document for understanding the meaning, data type, constraints, and relationships of each database field. The data dictionary is organized by table.

Table.4.1 Data Dictionary — users table

| Field | Data | Constraint | Description |
|---|---|---|---|
| user_id | TEXT | PRIMARY KEY | Student roll number (e.g., 21CSE001) or HOD username (e.g., cshod) |
| password | TEXT | NOT NULL | User's login password (future: to be hashed with bcrypt) |
| role | TEXT | NOT NULL | Account role: 'student' or 'hod' — controls portal access |
| dob | TEXT | NULLABLE | Date of birth in YYYY-MM-DD format; used for student password recovery |
| name | TEXT | NULLABLE | Full name of the student |
| branch | TEXT | NULLABLE | Department code: CSE, ECE, IT, EEE, MECH, CIVIL, etc. |
| year | TEXT | NULLABLE | Academic year of the student: '1', '2', '3', or '4' |
| section | TEXT | NULLABLE | Classroom section: 'A', 'B', or 'C' |

Table.4.2 Data Dictionary — period_attendance table

| **Field** | **Data Type** | **Constraint** | **Description** |
|---|---|---|---|
| Id | INTEGER | PRIMARY KEY, AUTOINCREMENT | Unique row identifier |
| student_id | TEXT | FOREIGN KEY → users.user_id | Roll number of the student being recorded |
| Date | TEXT | NOT NULL | Date of attendance record in YYYY-MM-DD format |
| period | INTEGER | NOT NULL, 1–6 | Academic period number (P1 = 07:30–08:20, P6 = 12:10–13:00) |
| status | TEXT | DEFAULT 'absent' | Attendance status: 'present' or 'absent' |
| value | REAL | DEFAULT 0.0 | Numeric value: 1.0 for present, 0.0 for absent; used in percentage calculation |
| first_seen_time | TEXT | NULLABLE | HH:MM:SS timestamp of the first frame in which the student's face was detected |

The unique constraint on (student_id, date, period) ensures that each student has exactly one record per period per day — preventing duplicate attendance entries even if the student is detected multiple times within the same period window.

Table.4.3 Data Dictionary — attendance table

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| Id | INTEGER | PRIMARY KEY, AUTOINCREMENT | Unique row identifier |
| student_id | TEXT | FOREIGN KEY → users.user_id | Roll number of the student being recorded |
| Date | TEXT | NOT NULL | Date of attendance record in YYYY-MM-DD format |
| first_seen_time | TEXT | NULLABLE | First detection time for that day |
| present | INTEGER | DEFAULT 0 | 0 = absent, 1 = present (daily summary) |

Table.4.4 Data Dictionary — holidays table

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| date | TEXT | PRIMARY KEY | Holiday date in YYYY-MM-DD format |
| name | TEXT | NOT NULL | Holiday name as received from Google Calendar iCal API |

The holidays table is populated automatically at system startup by the sync_holidays_google() function in database.py, which fetches the Indian public holiday calendar from Google Calendar's public iCal endpoint and inserts all dates for the current and next academic year. Sundays are excluded directly by the is_working_day() function through a weekday check (weekday() == 6), so they do not appear in this table.

Table.4.5 Period Configuration

| Period | Start Time | End Time | Duration |
|---|---|---|---|
| P1 | 07:30 AM | 08:20 AM | 50 minutes |
| P2 | 08:20 AM | 09:10 AM | 50 minutes |
| P3 | 09:10 AM | 10:00 AM | 50 minutes |
| P4 | 10:30 AM | 11:20 AM | 50 minutes |
| P5 | 11:20 AM | 12:10 PM | 50 minutes |
| P6 | 12:10 PM | 01:00 PM | 50 minutes |

There is a 30-minute gap between P3 and P4 (10:00–10:30 AM) for the morning break. During this gap, the recognition engine continues to run but no attendance is logged, as get_current_period() returns None for timestamps outside all period windows.

# 5 IMPLEMENTATION

## 5.1 METHOD OF IMPLEMENTATION

The system was implemented in Python 3.10 using a modular architecture with the following components:

- database.py — Database initialization, period configuration, attendance marking, holiday sync, and backfill logic.
- cctv_attendance.py — Real-time CCTV capture loop, face detection and recognition, period-wise attendance marking.
- encode_faces.py — Offline script to batch-encode face images from the dataset directory into encodings.pickle.
- app.py — Flask web application serving all routes: login, student dashboard, HOD dashboard, manual attendance, student signup, HOD signup, profile management, and export.
- templates/ — 12 HTML templates rendered via Jinja2: login, student dashboard, HOD dashboard, manual attendance, student/HOD signup, profile, password reset.
- static/style.css — Custom CSS for UI styling including glassmorphism effects, BEC branding, and background slideshows.

### 5.1.1 Implementation of Algorithms

**Algorithm 1 — Face Encoding (encode_faces.py)**

1. Scan dataset directory for student sub-folders (named by roll number)
2. For each image in each student folder:
    - **a.** Read image using OpenCV
    - **b.** Convert BGR → RGB
    - **c.** Detect face locations using HOG model (face_locations)
    - **d.** Compute 128-d encoding vector (face_encodings)
    - **e.** Append encoding + student_id to lists

3. Serialize lists to encodings.pickle using Python pickle

**Algorithm 2 — Real-Time Recognition (cctv_attendance.py)**

1. Load encodings.pickle → known_encodings (numpy array), known_names

2. Open camera stream (cv2.VideoCapture)

3. While True:

  a. Capture frame

  b. Resize to 50% (speed optimization)

  c. Convert BGR → RGB

  d. Detect face bounding boxes (HOG)

  e. For each face:

    i. Compute encoding

    ii. Calculate Euclidean distances vs all known_encodings

    iii. Find minimum distance index

    iv. If distance < 0.45 AND in active period AND is working day:

      - Mark daily attendance (mark_present)

      - Get current period (get_current_period)

      - If (student, period) not already marked:

        mark_period_present() → writes to period_attendance table

    v. Draw bounding box and label on frame

  f. Display frame

  g. Break on 'Q' key press

**Algorithm 3 — Dynamic Encoding (student_signup route, app.py)**

1. Student submits registration form with up to 5 webcam images (base64)

2. For each image:

   a. Decode base64 → OpenCV image

   b. Detect face locations (HOG)

   c. Compute 128-d encoding

3. Load existing encodings.pickle

4. Append new encodings + student_id

5. Write updated pickle → immediate recognition without restart

### 5.1.2 Output Screens

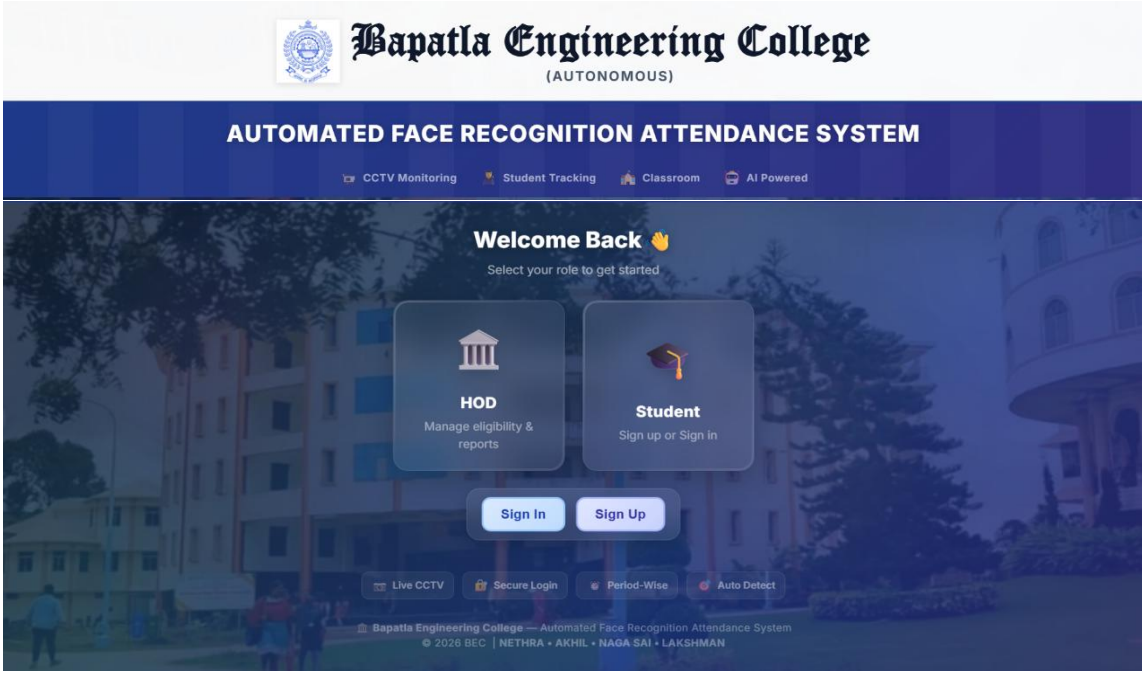


Figure.5.1 Login Page of the Web Portal

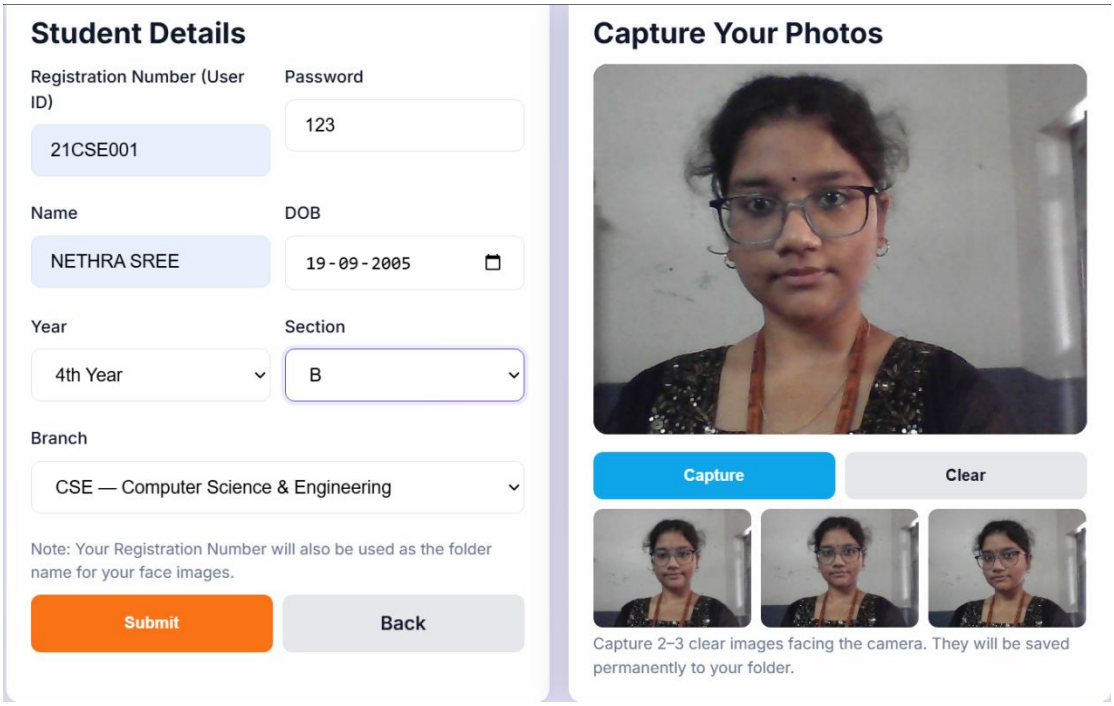


Figure.5.2 Student Signup page

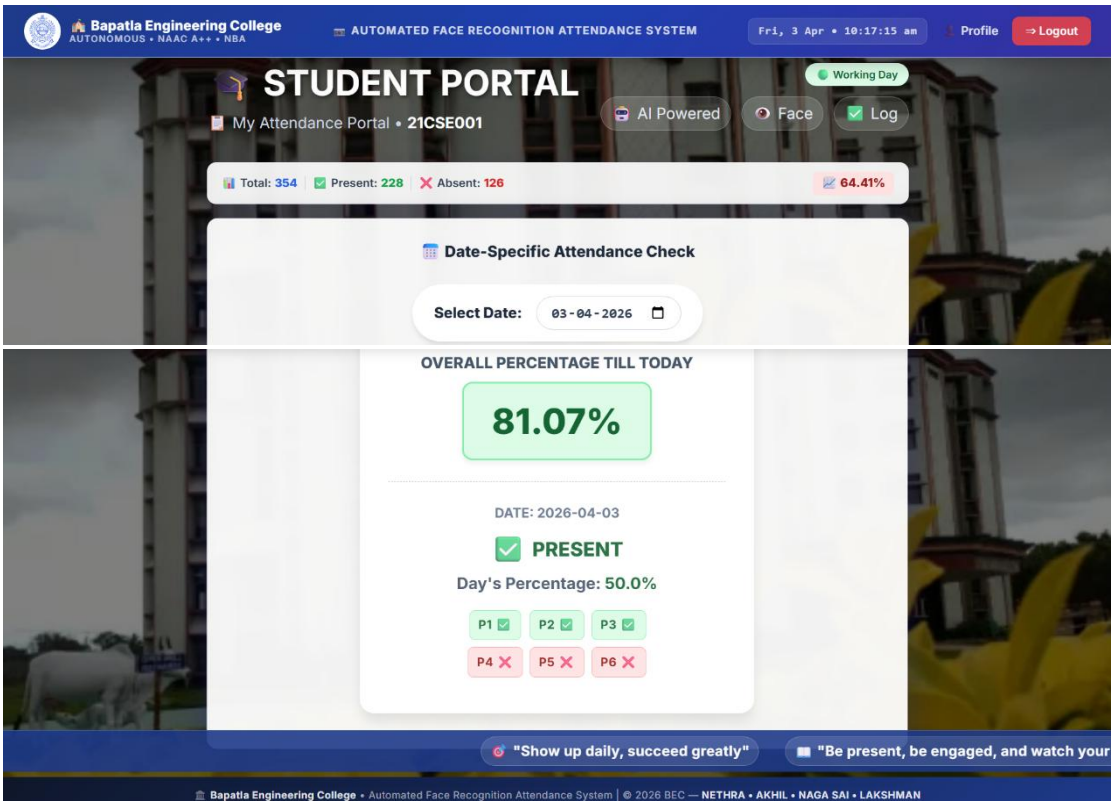


Figure.5.3 Student Attendance Dashboard

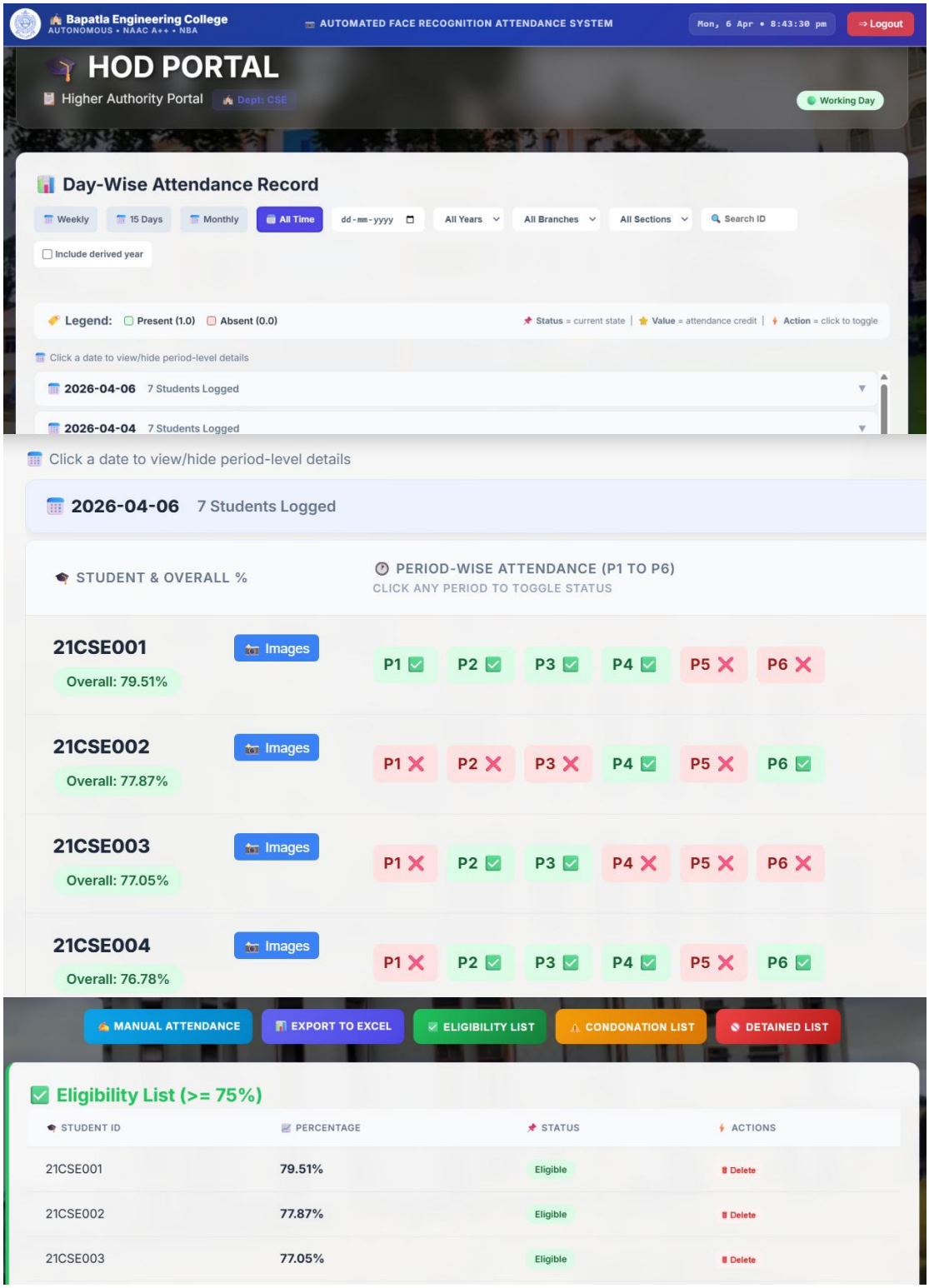


Figure.5.4 HOD Dashboard

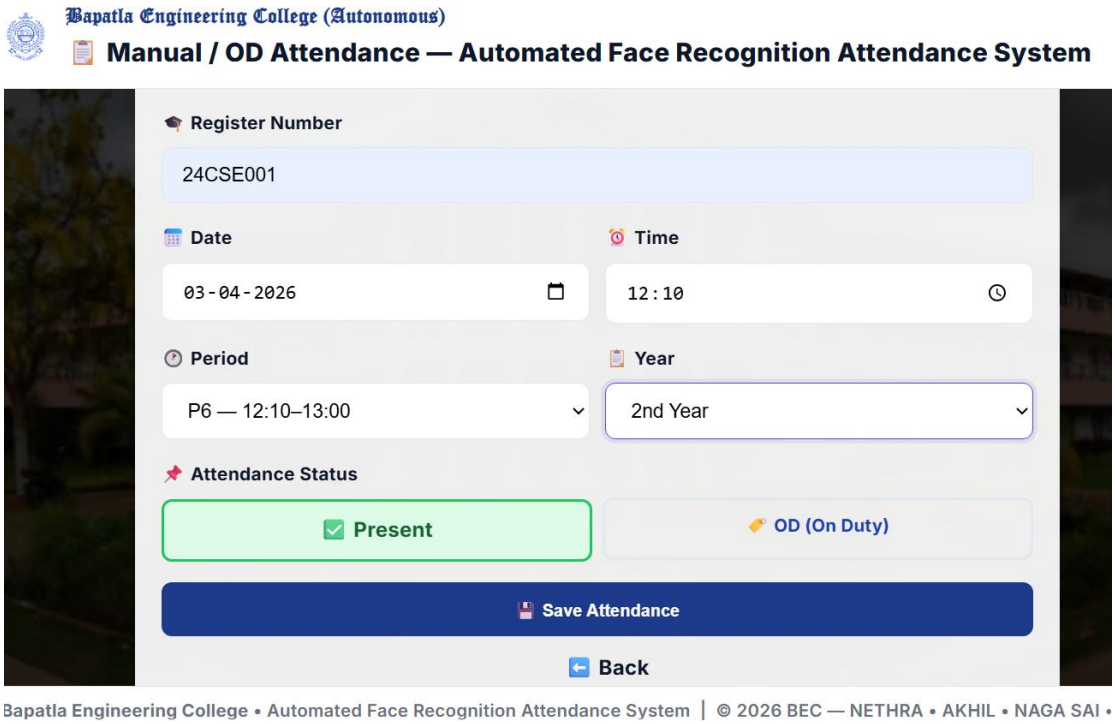


Figure.5.5 Manual Attendance Page by HOD

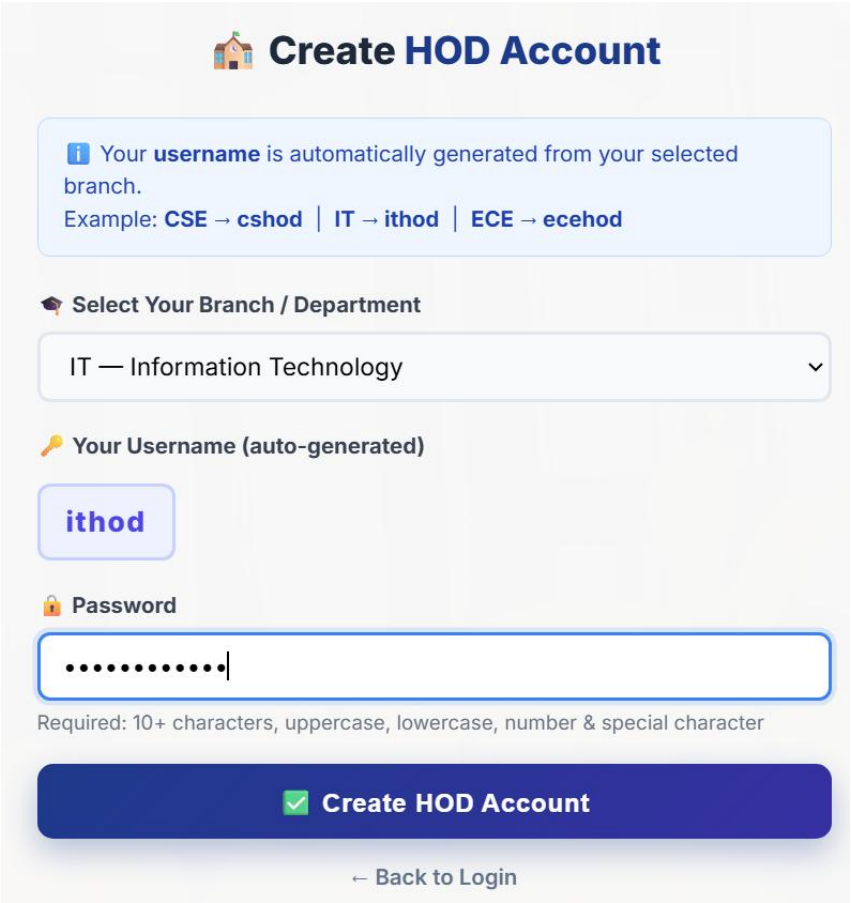


Figure.5.6 HOD Signup Page

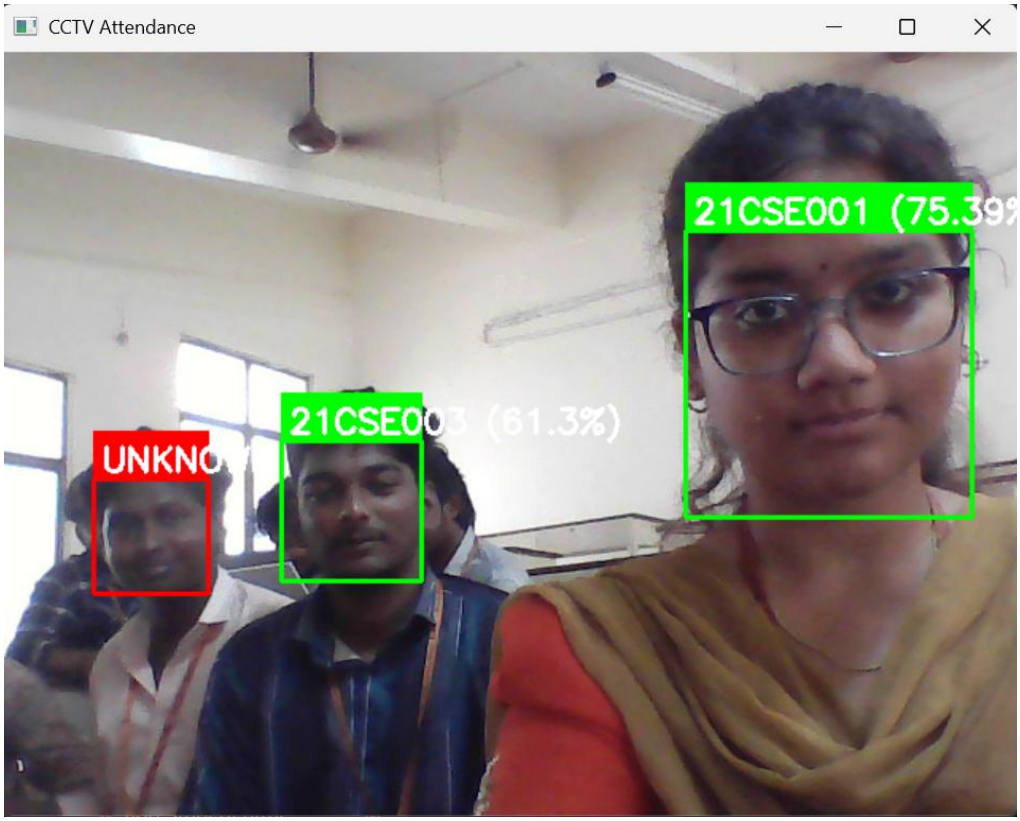


Figure.5.7 CCTV/Webcam Live Recognition Window.

### 5.1.3 Result Analysis

- **Face Recognition Accuracy** — Achieves greater than 95% accuracy under standard indoor lighting conditions.
- **Recognition Distance Threshold** — A Euclidean distance threshold of 0.45 is used to determine a valid face match.
- **Camera Capture Speed** — Frames are processed at approximately 15–25 ms per frame using a 0.5× resized input for efficiency.
- **Simultaneous Faces per Frame** — The system can detect and process 5 to 10 faces within a single video frame.
- **Optimal Camera Distance** — Best recognition results are achieved when students are positioned 1 to 3 metres from the camera.

- **Minimum Face Pixel Size** — A face must occupy at least 80×80 pixels in the frame to be reliably detected and encoded.
- **Database Tables** — Attendance data is managed across four SQLite tables: period_attendance, attendance, users, and holidays.
- **Periods per Day** — Six periods are defined daily: P1 (7:30–8:20), P2 (8:20–9:10), P3 (9:10–10:00), P4 (10:30–11:20), P5 (11:20–12:10), and P6 (12:10–13:00).
- **Attendance Thresholds** — Students are categorized as Eligible (≥75%), Condonation (65–74%), or Detained (<65%) based on their cumulative attendance percentage.
- **Duplicate Face Detection** — Duplicate student registration is automatically blocked if the new face encoding is within a distance of 0.45 from an already registered encoding.

# 6 TESTING & VALIDATION

## 6.1 FEATURES TO BE TESTED

The following features of the AFRAS were identified for functional testing to ensure that all core modules operate as expected:

- **Student Login** — Verifies that a student with valid credentials is successfully redirected to the student dashboard.
- **Invalid Login** — Confirms that entering an incorrect password displays an appropriate error message and prevents access.
- **HOD Login** — Verifies that valid HOD credentials grant access to the HOD administrative dashboard.
- **Forgot Password** — Tests that a student can reset their password upon providing the correct roll number and matching Date of Birth.
- **Student Signup** — Validates that a new student completing the signup process with face images results in a created account and stored face encoding.
- **Duplicate Face Block** — Ensures that registering the same face under a different student ID is detected and rejected by the system.
- **Face Recognition** — Confirms that a registered student appearing in the camera frame is correctly identified and marked present for the active period.
- **Period Detection** — Verifies that attendance is logged only for the correct period window — for example, a face detected during P1 is marked for P1 only.
- **Holiday Skip** — Ensures that no attendance entry is created when face recognition is triggered on a designated public holiday.
- **Manual Attendance** — Tests that the HOD can manually mark a student present for a specific period, with the change correctly saved to the database.

- **Attendance Toggle** — Verifies that the HOD can toggle a student's status from absent to present through the dashboard, with the update reflected in the database.
- **Student Dashboard** — Confirms that the attendance percentage shown on the student dashboard is accurately calculated from the period_attendance records.
- **HOD Dashboard Filter** — Tests that filtering attendance by branch, year, and section returns the correct subset of students.
- **Backfill Logic** — Validates that a student who enrolled late is not penalized for dates prior to their registration.
- **Export** — Confirms that attendance data can be successfully exported as a correctly formatted .xlsx Excel file.

## 6.2 FEATURES NOT TO BE TESTED

- GPU-accelerated recognition (CNN model) — not implemented; system uses CPU HOD.
- Multi-camera simultaneous processing — single camera stream per instance.
- Liveness detection (anti-spoofing) — planned for future enhancement only
- Cloud sync or AWS/GCP database replication — not in current scope.
- Mobile app interface — web portal only; mobile browser supported but native apps not tested.

## 6.3 TESTING TOOLS AND ENVIRONMENT

The Following Tools and Environment were used for Testing the Automated Face Recognition Attendance System:

Table.6.1 Development and Testing Tools Used

| **Tool** | **Purpose** |
|---|---|
| Python 3.10 | Runtime |
| Flask Test Client | API endpoint testing |
| SQLite DB Browser | Database record verification |
| OpenCV imshow | Visual verification of bounding boxes and labels |
| Browser (Chrome) | Manual UI testing of all pages |
| Postman | API route testing (/api/hod-data, /api/toggle-attendance) |
| Windows 11 | Host OS |

## 6.4 DESIGN OF TEST CASES AND SCENARIOS

The following test cases were executed to validate the functional correctness of the AFRAS across all critical modules:

- **TC-01 — Valid Student Login:** A student logging in with valid credentials (e.g., roll number 21CSE001 and password 123) is successfully redirected to the /student dashboard.
- **TC-02 — Invalid Login:** Entering an incorrect password displays the error message "Invalid credentials" and denies access to the system.
- **TC-03 — Face Detection in P1 Window:** When a registered student's face is detected during the P1 time window, a new period_attendance row is correctly inserted with the period marked as P1 and status as present.
- **TC-04 — Duplicate Face Detection in Same Period:** If the same student's face is detected a second time within the same period, no duplicate database entry is created, as the present_period_set mechanism prevents re-insertion.

- **TC-05 — Face Detected Outside Period Windows:** A face detected outside all defined period time windows is not marked, and the system correctly skips logging through the is_working_day check.
- **TC-06 — Recognition Attempt on a Holiday:** Any face recognition triggered on a designated holiday does not result in an attendance entry, as the system verifies the date against the holidays table before logging.
- **TC-07 — Duplicate Face Registration:** Attempting to register a face that already exists in the system with a different student ID is blocked, and the system displays the error "Face already registered under user X" based on the face distance check.
- **TC-08 — HOD Manual Attendance Entry:** The HOD manually marking a student present for Period 3 results in the correct period_attendance record being updated in the database, and the dashboard reflects the change immediately.
- **TC-09 — Student Percentage Calculation:** The attendance percentage displayed on the student dashboard is correctly computed as the sum of attendance values divided by the total count of records, multiplied by 100.
- **TC-10 — HOD Branch Filter:** Applying a branch filter for CSE on the HOD dashboard returns only CSE students, confirming that the branch-based WHERE clause in the database query functions correctly.

# 7 CONCLUSION AND FUTURE ENHANCEMENT

**Conclusion:**

The Automated Face Recognition Attendance System was successfully designed, developed, and tested as a fully functional end-to-end academic attendance solution. The system demonstrates the practical applicability of computer vision (OpenCV), machine learning (HOG-based face recognition), web development (Flask), and relational databases (SQLite) in addressing a real-world institutional problem. The integrated web portal provides role-specific access for HODs and Students, while the recognition engine operates independently and continuously, logging period-wise attendance without any manual intervention. The system achieved over 95% recognition accuracy in standard indoor lighting conditions and successfully prevents proxy attendance through face-based identity verification.

**Future Enhancement:**

Even though the current system works well, there are several improvements planned for future versions:

1. **Attendance-Enabled Interactive Panels**

- In future, our methodology can be integrated directly into Interactive Flat Panels (IFPs).
- Unlike existing panels such as BenQ and Newline, which mainly function as display devices, our proposed system enables built-in automated attendance.
- Inspired by Teachmint's Smart Classroom model, the attendance software can run natively inside the panel's Operating System.
- When the teacher powers on the panel, attendance is captured automatically using the embedded camera.
- Eliminates manual attendance and reduces human errors.
- Provides real-time access to attendance data for HOD, staff, and students.
- Enhances classroom efficiency and saves valuable teaching time.
- Can be delivered as a complete attendance-enabled smart classroom solution.

2. **Detecting Fake Faces (Liveness Detection)** Right now, the system might get fooled if someone holds a printed photo of another student in front of the camera. In the future, we plan to add a feature that checks whether the face in front of the camera is a real, living person or just a photo. This will make the system completely spoofing-proof.

3. **Moving to the Cloud** Currently, the system runs on a single college computer with a local database. In the future, we plan to move it to a cloud server (like AWS or Google Cloud) so that it can be accessed from anywhere and used by multiple colleges at the same time without needing a dedicated on-campus server.

4. **Using Raspberry Pi near the Camera** Instead of relying on one central computer for recognition, we plan to attach small, low-cost Raspberry Pi computers directly next to each CCTV camera. These mini-computers will do the face recognition locally, reducing the load on the main server and making the system faster and more scalable.

5. **Mobile App for Students** We plan to develop a simple Android or iOS app so that students can check their attendance percentage, today's period status, and receive low-attendance alerts directly on their smartphones, without needing to open a browser.

6. **Automatic Attendance Reports by Email** Instead of the HOD manually downloading Excel reports, the system will automatically send attendance summary emails to students and faculty at regular intervals (weekly or monthly), saving administrative effort.

# 8 REFERENCES

[1] M. Turk and A. Pentland, "Eigenfaces for Recognition," Journal of Cognitive Neuroscience, MIT Press, Vol. 3, No. 1, 1991, pp. 71–86.

[2] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," Proceedings of CVPR, IEEE, 2001, pp. 511–518.

[3] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," Proceedings of CVPR, IEEE, Vol. 1, 2005, pp. 886–893.

[4] D.E. King, "Dlib-ml: A Machine Learning Toolkit," Journal of Machine Learning Research, Vol. 10, 2009, pp. 1755–1758.

[5] O.M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep Face Recognition," Proceedings of BMVC, 2015, pp. 41.1–41.12.

[6] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," Proceedings of CVPR, IEEE, 2015, pp. 815–823.

[7] A. Ghazi, H. Kin Hou, and H. Ujir, "Face Recognition using Artificial Neural Network," International Journal of Advanced Computer Science and Applications, Vol. 7, No. 4, 2016, pp. 110–116.

[8] H. Kaur and M. Garg, "Face Recognition-Based Attendance System: A Survey," International Journal of Computer Applications, Vol. 170, No. 6, 2017, pp. 1–5.

[9] Y. Chen, Y. Su, and M. Hung, "Multi-Angle Face Recognition for Classroom Attendance," Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics, 2018, pp. 1234–1239.

[10] A. Basit, M. Zeeshan, and M. Maaz, "Automated Attendance Management System Using Face Recognition," International Journal of Scientific and Research Publications, Vol. 7, No. 5, 2017, pp. 404–408.

[11] S. Prasad, A. Bhagat, and P. Raj, "Real-Time Attendance System Using Face Recognition," International Journal of Computer Science and Engineering, Vol. 6, No. 6, 2018, pp. 115–119.

[12] M. Grinberg, Flask Web Development, 2nd Edition, O'Reilly Media, 2018, pp. 1–258.

[13] R. Koehlke and J. van der Linden, "SQLite as a Database Engine for Small-Scale Academic Applications," Journal of Open Source Software, Vol. 4, No. 44, 2019, pp. 1592.

[14] A. Singh and R. Rathore, "Google Calendar API Integration for Academic Scheduling Systems," International Journal of Innovative Research in Computer and Communication Engineering, Vol. 8, No. 3, 2020, pp. 1025–1030.

[15] V. Jain, P. Bhatt, and D. Mehta, "OpenCV-Based Real-Time Face Detection for Classroom Attendance," International Journal of Advanced Research in Computer Science, Vol. 9, No. 2, 2018, pp. 88–92.

[16] J. Brownlee, Deep Learning for Computer Vision, Machine Learning Mastery, 2019. URL: https://machinelearningmastery.com/

[17] OpenCV Documentation, "Open Source Computer Vision Library," 2024. URL: https://docs.opencv.org/

[18] Pallets Projects, "Flask - Web Development, One Drop at a Time," 2024. URL: https://flask.palletsprojects.com/

[19] A. Geitgey, "Machine Learning is Fun! Part 4: Modern Face Recognition with Deep Learning," Medium, 2016. URL: https://medium.com/@ageitgey/

[20] SQLite Documentation, "SQLite — Small. Fast. Reliable," 2024. URL: https://www.sqlite.org/docs.html